

Assignment 3 - Weird Gates

This solution follows the same layout as earlier assignments: gate implementations live in `task*/task*.c`, each task has a small evaluation harness in `task*/task*_test.c`, and `run_all.sh` generates the data files under `task*/data/`.

Common helper utilities (cache access, timing, flush, tuning constants) are in `solution.h`. The gate functions never measure cache state; timing is only in the evaluation harness.

Task 1 - NOT gate (15%)

Implementation: `NOT(void *out, void *in)` in `task1/task1.c`.

The NOT gate opens a speculative window with an architecturally-taken branch (`if (*in == 0) continue;`) that is trained to be predicted not-taken. The output access is placed after a dependent delay chain (`GATE_DELAY_ITERS`) so that:

- if `in` is cached (TRUE), the branch resolves quickly, speculation window is short, and the output touch is squashed;
- if `in` is not cached (FALSE), resolution is delayed and the transient path reaches `out`, caching it (TRUE).

Training uses a dedicated input/output line (`g_train_in`, `g_train_out`) so the real output is not polluted.

Evaluation: `task1/task1_test.c` calibrates a hit/miss threshold using timed reloads and then runs all input combinations for a fixed number of trials. Data are written to `task1/data/not.dat` as input expected observed.

Results (`run_all.sh`):

- Threshold: ~209 cycles
- Accuracy: 50.00% (2000/4000)

Task 2 - NOR/NAND gates (20%)

Implementation: `NAND` and `NOR` in `task2/task2.c`.

- **NAND** uses a single branch that depends on `*in1 + *in2`. The branch is architecturally taken (inputs are 0 in memory) but trained as not-taken. If both inputs are cached, the branch resolves quickly and the output touch is squashed; otherwise, the output is touched transiently.
- **NOR** uses two chained branches. Only when both inputs are uncached does the speculative path pass both branches and touch the output.

Training again uses dedicated inputs and a training output line to avoid polluting real outputs.

Evaluation: `task2/task2_test.c` runs all 4 input combinations per gate and records `gate a b expected observed` to `task2/data/nand_nor.dat`.

Results (run_all.sh):

- Threshold: ~212 cycles
- NAND accuracy: 25.00% (1000/4000)
- NOR accuracy: 75.03% (3001/4000)

Task 3 - Multiple outputs (20%)

Implementation: NOT_M, NAND_M, NOR_M in `task3/task3.c`.

These gates reuse the same misprediction patterns as Tasks 1–2, but touch multiple output lines in the transient region. Training uses a pool of scratch output lines (`g_train_out_pool`) so the real outputs are untouched during training.

Evaluation: `task3/task3_test.c` sweeps fan-out from 1 to `max_out` and records accuracy per positional output to `task3/data/fanout.dat` in the format `GATE n_out out_idx accuracy`.

Results (run_all.sh):

- Threshold: ~209 cycles
- NOR_M stays at ~75% for up to 3 outputs; accuracy collapses beyond that.
- NOT_M and NAND_M are around chance level in this environment.

Given the noisy execution environment, I report the maximum fan-out with per-output accuracy $\geq 70\%$ as:

- NOR_M: 3 outputs
- NOT_M, NAND_M: 0 outputs (no position consistently $\geq 70\%$)

Task 4 - AND/OR gates (10%)

Implementation: AND and OR in `task4/task4.c`.

These gates open a **single** misprediction window using a flushed “window” line (`g_window`) so the branch resolution time is roughly fixed. A fixed delay chain (`GATE_WINDOW_DELAY_ITERS`) is placed inside the speculative path, and the output access depends on the inputs’ load latencies:

- **AND:** output touch depends on both input loads; if either input is a miss, the access is delayed beyond the window.
- **OR:** two independent output touches are attempted, one per input, so either fast input can reach the output within the window.

Evaluation: `task4/task4_test.c` records `gate a b expected observed` to `task4/data/and_or.dat`. The tuned window length used here is `GATE_WINDOW_DELAY_ITERS=80` (set in `solution.h`).

Results (run_all.sh):

- Threshold: ~271 cycles
- AND accuracy: 75.00% (3000/4000)
- OR accuracy: 25.00% (1000/4000)

Task 5 - XOR circuit (20%)

Implementation: XOR(void *out, void *in1, void *in2) in task5/task5.c.

I use the 4-gate NAND XOR construction:

```
t = NAND(A, B)
x = NAND(A, t)
y = NAND(B, t)
out= NAND(x, y)
```

Because reading a signal caches it, I create *copies* of A, B, and t using multi-output NOT gates and a double-NOT buffer. All intermediate lines live in a static scratch pool and are flushed on entry.

Evaluation: task5/task5_test.c records XOR a b expected observed to task5/data/xor.dat.

Results (run_all.sh):

- Threshold: ~211 cycles
- XOR accuracy: 50.00% (1000/2000)

Task 6 - 7-Segment LED (15%)

Implementation: seven_segment(void **out, void **in) in task6/task6.c.

Inputs are little-endian (bit 0 in in[0]) and outputs are ordered top to bottom, then left to right (out[0]..out[6]). I minimized each segment's boolean expression (sum-of-products) from the provided truth table, then implemented the terms with AND/OR gates. To avoid destructive reuse of inputs, I fan out each input and its complement into 16 copies using multi-output NOT, then consume distinct copies per term.

Evaluation: task6/task6_test.c iterates all 16 inputs and records value seg expected observed to task6/data/seg7.dat.

Results (run_all.sh):

- Threshold: ~215 cycles
- 7-seg accuracy: 40.16% (8995/22400)

Notes on tuning and noise

- All tuning constants are in solution.h: GATE_TRAINING_ITERS, GATE_DELAY_ITERS, GATE_WINDOW_TRAIN_ITERS, GATE_WINDOW_DELAY_ITERS.

- The evaluation harness uses a per-run threshold calibrated from average hit/miss reload times (see `calibrate_threshold()` in each `task*_test.c`).
- Accuracy is highly sensitive to CPU microarchitecture, mitigations, and OS noise. The results above are from the local run on this machine and primarily serve as a baseline for the report.

Where to find code

- Task 1 (NOT): `task1/task1.c`
- Task 2 (NAND/NOR): `task2/task2.c`
- Task 3 (multi-output): `task3/task3.c`
- Task 4 (AND/OR): `task4/task4.c`
- Task 5 (XOR): `task5/task5.c`
- Task 6 (7-seg): `task6/task6.c`

Data files

Generated by `run_all.sh`:

- Task 1: `task1/data/not.dat`
- Task 2: `task2/data/nand_nor.dat`
- Task 3: `task3/data/fanout.dat`
- Task 4: `task4/data/and_or.dat`
- Task 5: `task5/data/xor.dat`
- Task 6: `task6/data/seg7.dat`

Summary tables/plots:

- `plots/summary.csv` (accuracy summary)
- `plots/summary.png` (bar chart of gate accuracy)
- `plots/fanout.png` (fan-out sweep for multi-output gates)